ASSIGNMENT-11.5

NAME: TEENESWARI

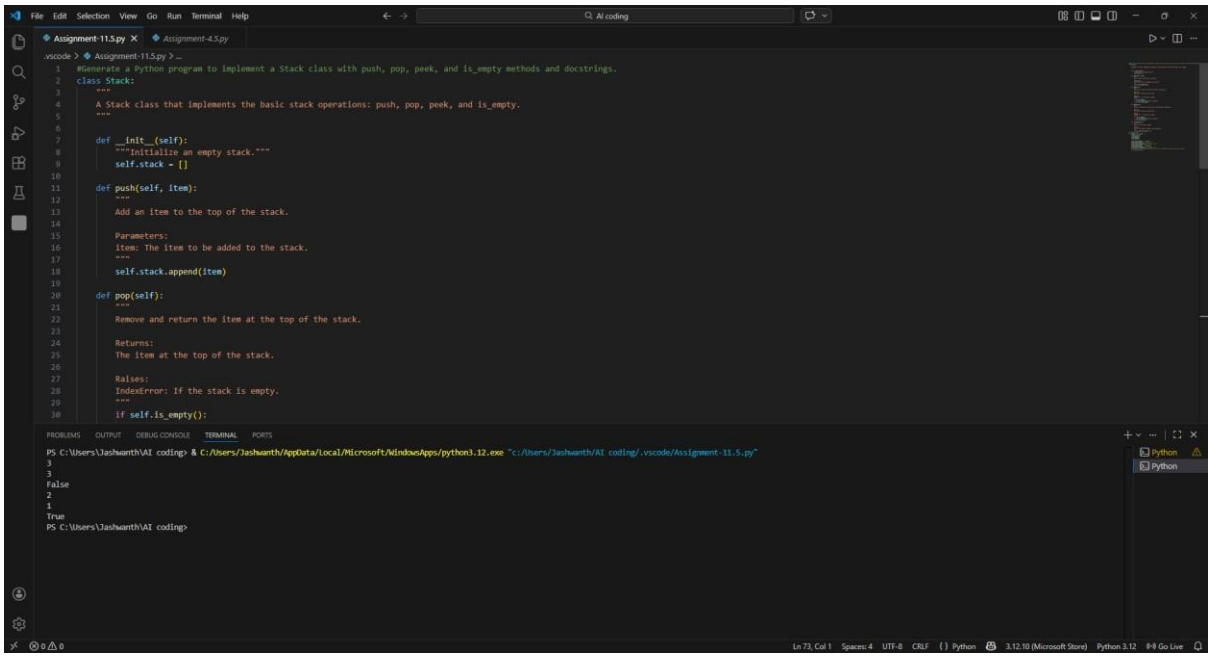
ROLLNO: 2303A51932

BATCH: 30

TASK-1

PROMPT: Generate a Python program to implement a Stack class with push, pop, peek, and is_empty methods and show example usage.

CODE:



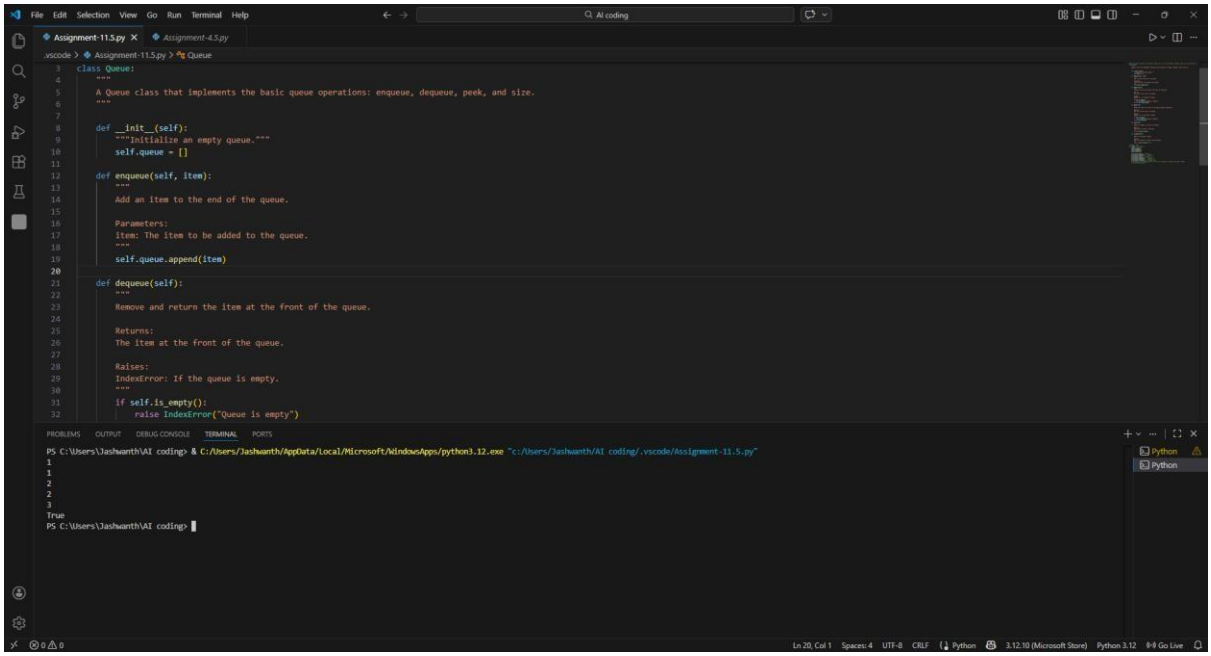
OBSERVATION:

The stack was implemented successfully using a class and Python list. The operations push, pop, peek, and is_empty worked correctly following the **LIFO (Last In First Out)** principle. The most recently added element was removed first when pop was executed.

TASK-2

PROMPT: Create a Python program to implement a Queue using a list with enqueue, dequeue, peek, and size operations.

CODE:



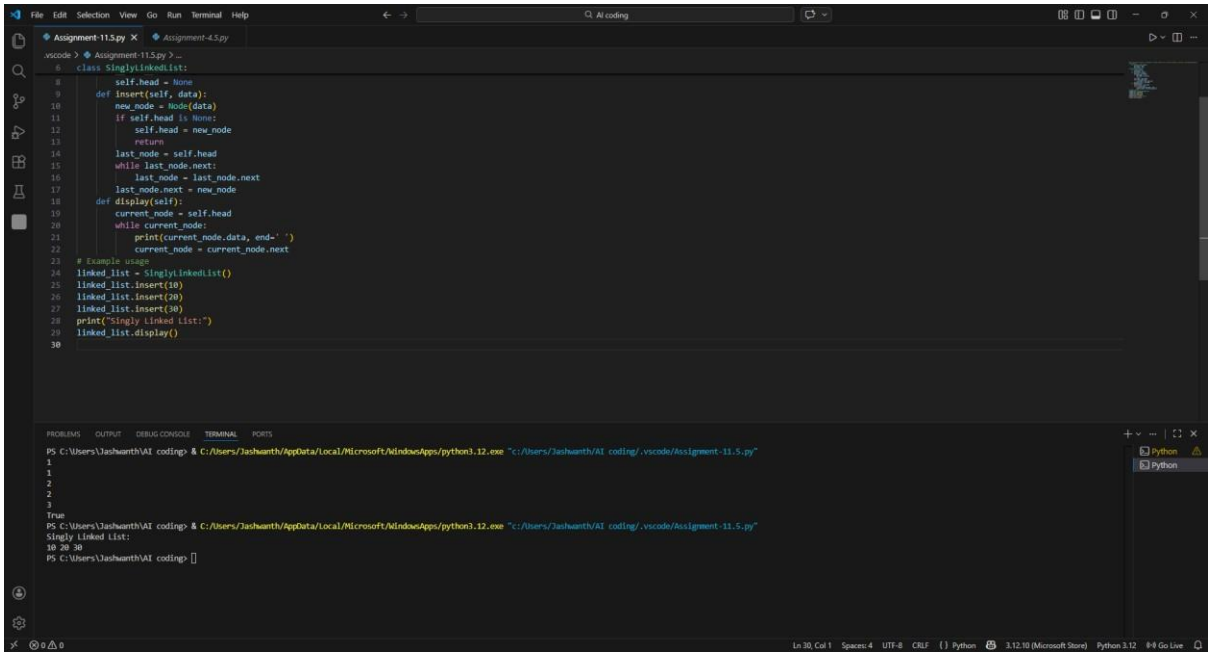
OBSERVATION:

The queue program worked correctly using a list to store elements. The operations enqueue, dequeue, peek, and size were implemented successfully. The queue followed the **FIFO (First In First Out)** principle where the first inserted element was removed first.

TASK-3

PROMPT: Write a Python program to implement a singly linked list with node creation, insertion, and display methods.

CODE:



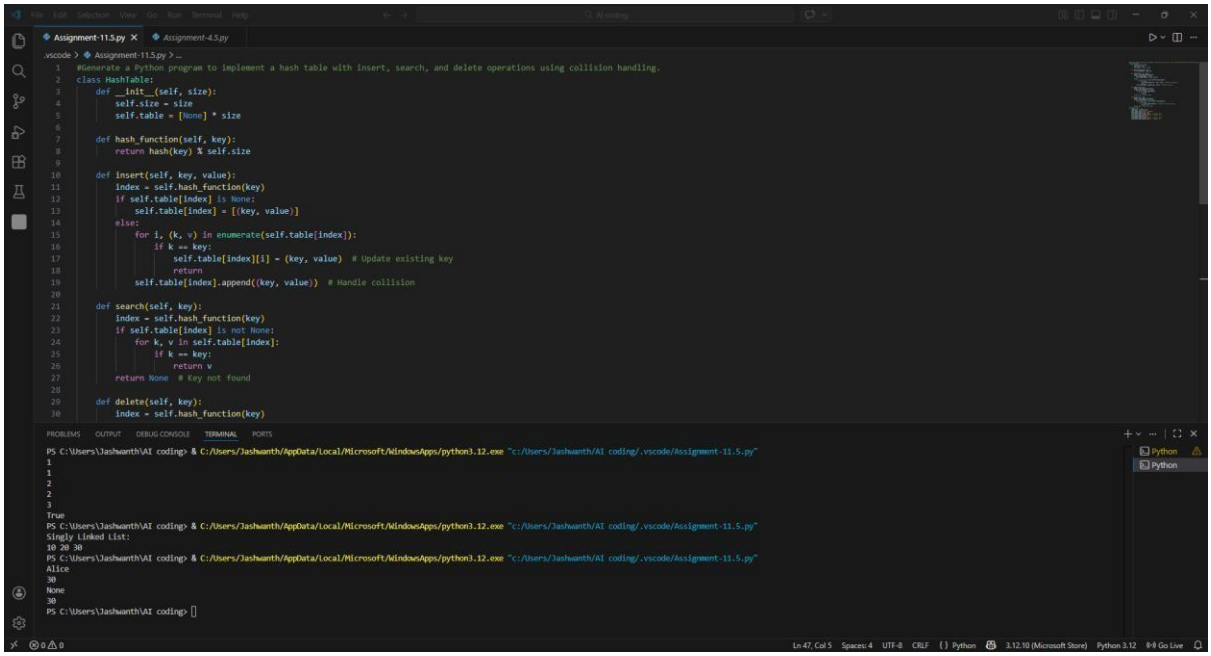
OBSERVATION:

The singly linked list was created using nodes connected through references. New nodes were inserted successfully, and the display function traversed the list and printed all elements. This demonstrated how dynamic memory allocation works in linked lists.

TASK-4

PROMPT: Generate a Python program to implement a hash table with insert, search, and delete operations using collision handling.

CODE:



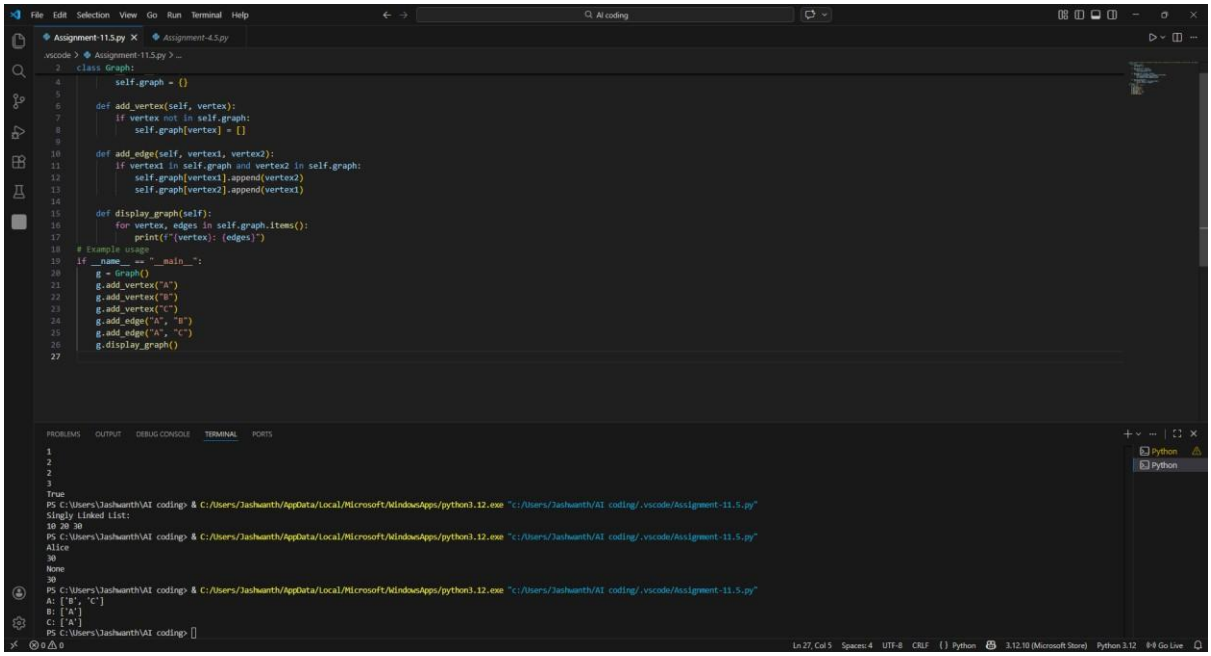
OBSERVATION:

The hash table successfully stored key-value pairs using a hashing function. Collision handling was implemented using chaining. The insert, search, and delete operations worked efficiently, showing how hash tables provide fast data retrieval.

TASK-5

PROMPT: Create a Python program to represent a graph using an adjacency list with methods to add vertices, add edges, and display the graph.

CODE:



```python
class Graph:
        self.graph = {}

    def add_vertex(self, vertex):
        if vertex not in self.graph:
            self.graph[vertex] = []

    def add_edge(self, vertex1, vertex2):
        if vertex1 in self.graph and vertex2 in self.graph:
            self.graph[vertex1].append(vertex2)
            self.graph[vertex2].append(vertex1)

    def display_graph(self):
        for vertex, edges in self.graph.items():
            print(f"{vertex}: {edges}")
# Example usage
if __name__ == "__main__":
    g = Graph()
    g.add_vertex("A")
    g.add_vertex("B")
    g.add_vertex("C")
    g.add_edge("A", "B")
    g.add_edge("A", "C")
    g.display_graph()
```

```
1
2
2
3
True
PS C:\Users\Jashwanth\AI coding> & C:/Users/Jashwanth/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/Jashwanth/AI coding/.vscode/Assignment-11.5.py"
Singly Linked List:
10 20 30
PS C:\Users\Jashwanth\AI coding> & C:/Users/Jashwanth/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/Jashwanth/AI coding/.vscode/Assignment-11.5.py"
Alice
30
None
30
PS C:\Users\Jashwanth\AI coding> & C:/Users/Jashwanth/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/Jashwanth/AI coding/.vscode/Assignment-11.5.py"
A: ['B', 'C']
B: ['A']
C: ['A']
PS C:\Users\Jashwanth\AI coding>
```

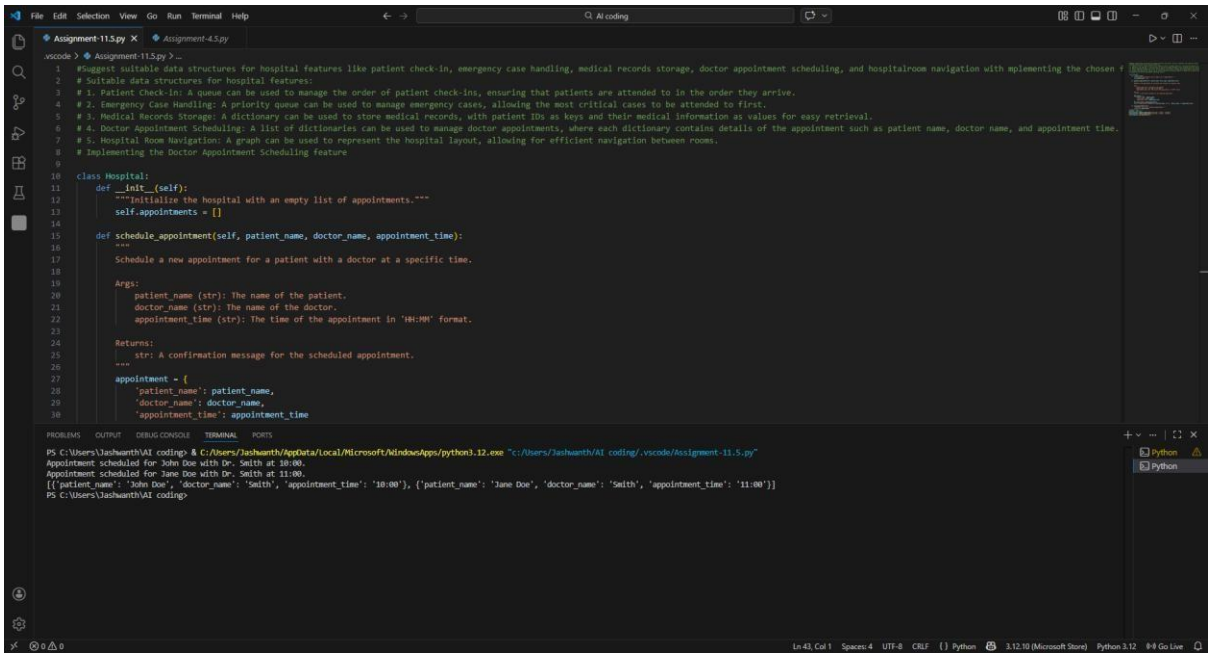
OBSERVATION:

The graph was represented using an adjacency list. Vertices and edges were added correctly, and the graph structure was displayed properly. This showed how graphs can represent relationships or connections between different nodes.

TASK-6

PROMPT: Suggest suitable data structures for hospital features like patient check-in, emergency handling, medical records, appointment scheduling, and room navigation with justification.

CODE:



```python
#Suggest suitable data structures for hospital features like patient check-in, emergency case handling, medical records storage, doctor appointment scheduling, and hospitalroom navigation with mplementing the chosen f
# Suitable data structures for hospital features:
# 1. Patient Check-in: A queue can be used to manage the order of patient check-ins, ensuring that patients are attended to in the order they arrive.
# 2. Emergency Case Handling: A priority queue can be used to manage emergency cases, allowing the most critical cases to be attended to first.
# 3. Medical Records Storage: A dictionary can be used to store medical records, with patient IDs as keys and their medical information as values for easy retrieval.
# 4. Doctor Appointment Scheduling: A list of dictionaries can be used to manage doctor appointments, where each dictionary contains details of the appointment such as patient name, doctor name, and appointment time.
# 5. Hospital Room Navigation: A graph can be used to represent the hospital layout, allowing for efficient navigation between rooms.
# Implementing the Doctor Appointment Scheduling feature

class Hospital:
    def __init__(self):
        """Initialize the hospital with an empty list of appointments."""
        self.appointments = []

    def schedule_appointment(self, patient_name, doctor_name, appointment_time):
        """
        Schedule a new appointment for a patient with a doctor at a specific time.

        Args:
            patient_name (str): The name of the patient.
            doctor_name (str): The name of the doctor.
            appointment_time (str): The time of the appointment in 'HH:MM' format.

        Returns:
            str: A confirmation message for the scheduled appointment.
        """
        appointment = {
            'patient_name': patient_name,
            'doctor_name': doctor_name,
            'appointment_time': appointment_time
```

```
PS C:\Users\Jashwanth\AI coding> & C:/Users/Jashwanth/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/Jashwanth/AI coding/.vscode/Assignment-11.5.py"
Appointment scheduled for John Doe with Dr. Smith at 10:00.
Appointment scheduled for Jane Doe with Dr. Smith at 11:00.
[{'patient_name': 'John Doe', 'doctor_name': 'Smith', 'appointment_time': '10:00'}, {'patient_name': 'Jane Doe', 'doctor_name': 'Smith', 'appointment_time': '11:00'}]
PS C:\Users\Jashwanth\AI coding>
```

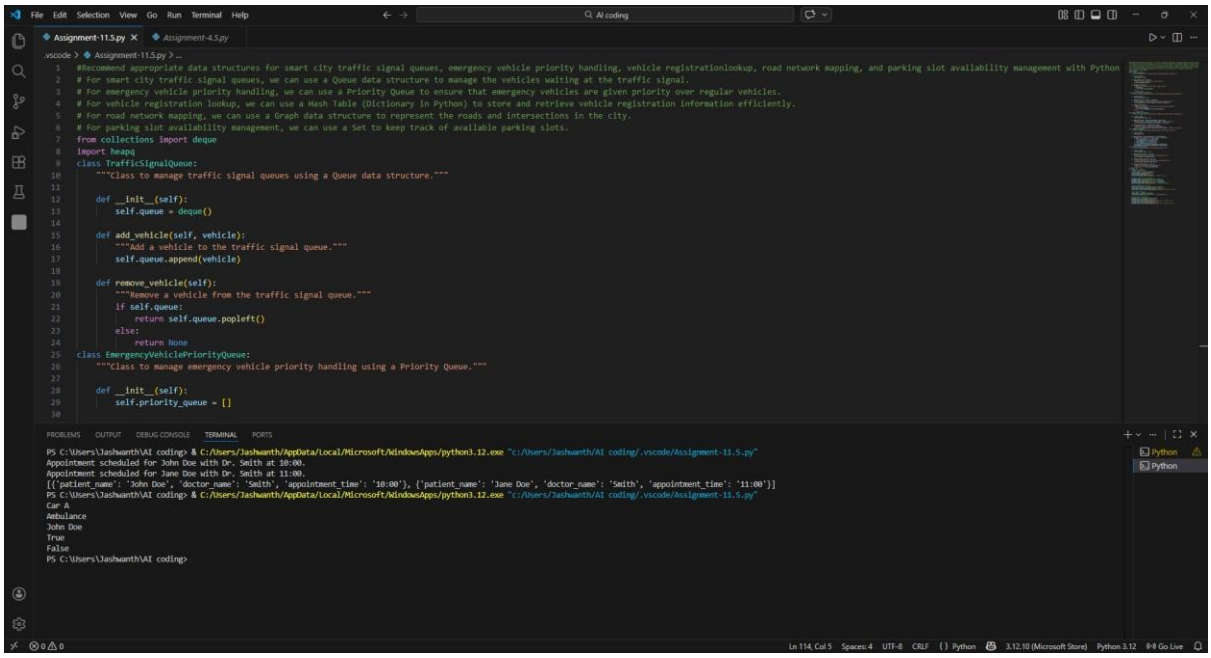
OBSERVATION:

Different data structures were selected for different hospital operations. For example, queues can manage patient check-ins, priority queues can handle emergency cases, dictionaries can store medical records, and graphs can represent hospital navigation. The appointment scheduling feature worked correctly.

TASK-7

PROMPT: Recommend appropriate data structures for traffic signal queues, emergency vehicle priority, vehicle lookup, road mapping, and parking management.

CODE:



```python
#Recommend appropriate data structures for smart city traffic signal queues, emergency vehicle priority handling, vehicle registrationlookup, road network mapping, and parking slot availability management with Python
# For smart city traffic signal queues, we can use a Queue data structure to manage the vehicles waiting at the traffic signal.
# For emergency vehicle priority handling, we can use a Priority Queue to ensure that emergency vehicles are given priority over regular vehicles.
# For vehicle registration lookup, we can use a Hash Table (Dictionary in Python) to store and retrieve vehicle registration information efficiently.
# For road network mapping, we can use a Graph data structure to represent the roads and intersections in the city.
# For parking slot availability management, we can use a Set to keep track of available parking slots.
from collections import deque
import heapq
class TrafficSignalQueue:
    """Class to manage traffic signal queues using a Queue data structure."""

    def __init__(self):
        self.queue = deque()

    def add_vehicle(self, vehicle):
        """Add a vehicle to the traffic signal queue."""
        self.queue.append(vehicle)

    def remove_vehicle(self):
        """Remove a vehicle from the traffic signal queue."""
        if self.queue:
            return self.queue.popleft()
        else:
            return None
class EmergencyVehiclePriorityQueue:
    """Class to manage emergency vehicle priority handling using a Priority Queue."""

    def __init__(self):
        self.priority_queue = []
```

```
PS C:\Users\Jashwanth\AI coding> & C:/Users/Jashwanth/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/Jashwanth/AI coding/.vscode/Assignment-11.5.py"
Appointment scheduled for John Doe with Dr. Smith at 10:00.
Appointment scheduled for Jane Doe with Dr. Smith at 11:00.
[{'patient_name': 'John Doe', 'doctor_name': 'Smith', 'appointment_time': '10:00'}, {'patient_name': 'Jane Doe', 'doctor_name': 'Smith', 'appointment_time': '11:00'}]
PS C:\Users\Jashwanth\AI coding> & C:/Users/Jashwanth/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/Jashwanth/AI coding/.vscode/Assignment-11.5.py"
Car A
Ambulance
John Doe
True
False
PS C:\Users\Jashwanth\AI coding>
```

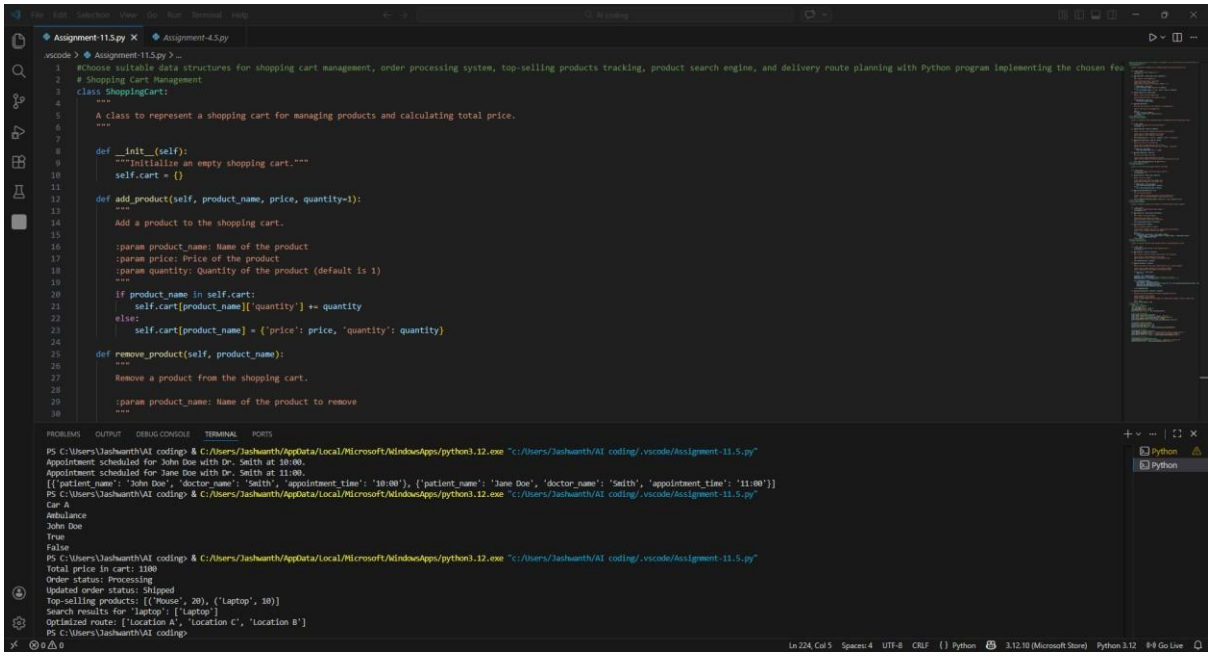
OBSERVATION:

Appropriate data structures were used for traffic management features. Queues handled traffic signals, priority queues managed emergency vehicles, dictionaries stored vehicle registration data, graphs represented road networks, and sets tracked parking availability. The implementation demonstrated efficient handling of real-world traffic scenarios.

TASK-8

PROMPT: Choose suitable data structures for shopping cart management, order processing, top-selling products tracking, product search, and delivery route planning.

CODE:



```
#Choose suitable data structures for shopping cart management, order processing system, top-selling products tracking, product search engine, and delivery route planning with Python program implementing the chosen fea
# Shopping Cart Management
class ShoppingCart:
    """
    A class to represent a shopping cart for managing products and calculating total price.
    """

    def __init__(self):
        """Initialize an empty shopping cart."""
        self.cart = {}

    def add_product(self, product_name, price, quantity=1):
        """
        Add a product to the shopping cart.

        :param product_name: Name of the product
        :param price: Price of the product
        :param quantity: Quantity of the product (default is 1)
        """
        if product_name in self.cart:
            self.cart[product_name]['quantity'] += quantity
        else:
            self.cart[product_name] = {'price': price, 'quantity': quantity}

    def remove_product(self, product_name):
        """
        Remove a product from the shopping cart.

        :param product_name: Name of the product to remove
        """
```

```
PS C:\Users\Jashwanth\AI coding> & C:/Users/Jashwanth/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/Jashwanth/AI coding/.vscode/Assignment-11.5.py"
Appointment scheduled for John Doe with Dr. Smith at 10:00.
Appointment scheduled for Jane Doe with Dr. Smith at 11:00.
[{'patient_name': 'John Doe', 'doctor_name': 'Smith', 'appointment_time': '10:00'}, {'patient_name': 'Jane Doe', 'doctor_name': 'Smith', 'appointment_time': '11:00'}]
PS C:\Users\Jashwanth\AI coding> & C:/Users/Jashwanth/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/Jashwanth/AI coding/.vscode/Assignment-11.5.py"
Car A
Ambulance
John Doe
True
False
PS C:\Users\Jashwanth\AI coding> & C:/Users/Jashwanth/AppData/Local/Microsoft/WindowsApps/python3.12.exe "c:/Users/Jashwanth/AI coding/.vscode/Assignment-11.5.py"
Total price in cart: 1100
Order status: Processing
Updated order status: Shipped
Top-selling products: [('Mouse', 20), ('Laptop', 10)]
Search results for 'laptop': ['Laptop']
Optimized route: ['Location A', 'Location C', 'Location B']
PS C:\Users\Jashwanth\AI coding>
```

OBSERVATION:

Different data structures were used to manage e-commerce operations. Dictionaries stored cart items and orders, sorting was used to find top-selling products, and search functionality helped locate products quickly. The system demonstrated how data structures help manage large-scale online shopping operations efficiently.